

Agent Foundations:

Reasoning Loops, Tool Calling & Raw Python Agents

Goal

Build a minimal agent loop from scratch in raw Python, no framework, against an OpenAI-compatible chat completions API. The point was to understand what a while-loop with tool calling actually does before relying on LangChain or LangGraph to hide it.

Note on API provider: OpenRouter was the first choice because it exposes many tool-calling-capable models behind one OpenAI-compatible endpoint. During testing its free-tier daily request limit was hit quickly, so the notebook's active configuration was switched to Groq's llama-3.3-70b-versatile endpoint, with the OpenRouter and NVIDIA alternatives left commented out in the setup cell. The agent code itself is provider-agnostic: switching is a one-line `base_url` and `api_key` change, and every observation below applies to either provider unless explicitly flagged as free-tier-specific.

The ReAct Loop

An agent is a chatbot given tools and put in a loop. Instead of answering in one shot, it cycles through Reason (decide what it needs next), Act (call a tool), and Observe (read what the tool returned), folding each observation into the next round of reasoning. The loop ends when the model has enough information and returns plain text instead of another tool call. In the implementation this is a while-loop: send the conversation, check `response.finish_reason`, execute the tool if one was requested, append both the model's tool call and the tool's result back into the message history, repeat. A `max_iterations` cap stops the loop if the model never converges on a final answer.

Tool Schemas Used

Two tools were registered: `calculator`, which evaluates a literal arithmetic expression and returns the numeric result, and `get_weather`, a stub returning temperature and condition for Perth, Karachi, Hobart, and Melbourne. Each schema follows the OpenAI function-calling format the API expects: `type: function`, wrapping name, description, and a parameters JSON Schema. Descriptions state what the tool does, what each argument means, its expected format, and what it returns, since the model never sees the tool's source code and a vague description leads directly to wrong tool choices or malformed arguments.

Issues Found During Testing

Live testing surfaced several problems the first version did not handle. First, `calculator` had no input validation; an unvalidated eval would crash outright on a malformed or malicious expression instead of failing gracefully. Fixed with an AST-based whitelist: the expression is parsed with `ast.parse`, then walked recursively through `_safe_eval`, allowing only numeric constants, the four binary operators (+, -, *, /), unary signs, and parentheses; division by zero, overlong inputs, and any unsupported AST node raise a `ValueError` with a clear message. Second, no failure test covered a request that matched neither registered tool, so a stock-price prompt and a file-read prompt were added as genuine out-of-scope cases, both of which the model correctly declined. Third, and most significant: an earlier run returned a fabricated weather answer for a city not in the stub data - the model had skipped the tool call entirely and answered from memory rather than calling `get_weather`; `finish_reason` came back stop with invented content instead of `tool_calls`. This is a hallucinated-answer failure mode distinct from a hallucinated tool call. Fixed with an explicit system prompt instructing the model to always call the matching tool for anything the tools cover, to ask the user for a city when one is missing rather than guessing, and to say so directly when no tool applies. The same system prompt also forbids emojis, a model default rather than anything in the code; there is no code-level post-processing, so the no-emoji guarantee depends on the model actually following the instruction.

Two further failure modes surfaced during the break-attempt tests that are worth flagging because they originate at the API rather than in the agent's own logic. Asking for the weather in Atlantis (a city not in the stub) caused the chat-completions API to reject the model's tool-call attempt with a `tool_use_failed` error before the agent's executor could run the tool - the model had produced a well-formed call, but the API side refused to dispatch it. And a tight `max_iterations=2` cap on a four-city weather ranking task hit the API's free-tier rate limit on the very first call, so the iteration cap was never actually exercised - the rate limit got there first. Both are free-tier-specific observations rather than architectural flaws; on a paid tier neither would occur.

Failure Modes Observed

Failure mode	Cause	Mitigation
Infinite / runaway loop	Model never returns a final answer, keeps requesting tools.	max_iterations hard cap, loop returns None instead of hanging.
Hallucinated tool call	Model requests a tool name that was never registered.	Executor checks the name against the registered tool map and raises a caught error.
Hallucinated answer, no tool call	Model answers from memory instead of calling the tool, observed live on a weather query for a city not in the stub.	System prompt instructs the model to always call the matching tool and never answer those from memory.
Invalid calculator input	Unvalidated eval on a malformed or malicious expression.	AST-based whitelist via ast.parse and a recursive _safe_eval that allows only numeric constants, +, -, *, /, unary signs, and parentheses; division by zero, overlong input, and unsupported nodes raise a clean ValueError.
Wrong / bad tool arguments	get_weather called with a city not in the stub data.	Tool raises a normal exception, caught and returned as an ERROR tool message so the model can recover next turn.
Out-of-scope request	Request needs neither registered tool (e.g. a stock price or a file-read task).	System prompt tells the model to say so directly instead of guessing.
Silent errors	A failed tool call could look identical to a correct one in the final answer.	Every reason, act, and observe step is logged, so failures are visible in the transcript.

Why Frameworks Exist

The raw loop, schemas, and executor total under 200 lines, and that already required care around max iterations, error handling, input validation, and getting the model to actually comply with tool use. Frameworks like LangChain, LangGraph, and CrewAI exist because real systems need many tools, multiple cooperating agents, long-running state, streaming output, and retries, and that plumbing is largely the same across projects. A framework packages it once so teams focus on what tools their agent needs rather than re-solving how to safely run a while-loop with tool calls each time. Having built it by hand first, including hitting the hallucinated-answer bug directly, means that when a framework's abstraction does something unexpected, the underlying mechanism is already understood.